

Architektury systemów komputerowych

Lista zadań nr 7

Na zajęcia 15 i 28 kwietnia 2026

Trochę dokładniejsze informacje na temat tego co było na wykładzie można znaleźć w [1, §5.1] – [1, §5.9].

Skompilowany kod najlepiej prezentować przy pomocy [Compiler Explorer](#)¹ używając najnowszej dostępnej wersji kompilatora GCC.

Zadanie 1. W tym zadaniu rozważamy procesor, w którym poszczególne fazy wykonywania rozkazów trwają odpowiednio: IF 300ps, ID 200ps, EX 250ps, MEM 600ps, WB 100ps. Jaka będzie długość cyklu zegarowego w przypadku procesora jednocyklowego, a jaka w przypadku procesora z przetwarzaniem potokowym? Załóżmy, że możemy rozdzielić jedną z faz na dwie mniejsze (każdą o czasie działania równym połowie wyjściowej fazy). Którą fazę najlepiej wybrać? Jak takie rozdzielenie wpłynie na długość cyklu? To samo zadanie rozwiąż przy założeniu, że fazy trwają odpowiednio: IF 200ps, ID 150ps, EX 120ps, MEM 190ps, WB 140ps.

Zadanie 2. Rozważmy następujące dwa fragmenty kodu:

1 mov (%rax), %rdi	1 add %rcx, %rdi
2 add %rdi, %rsi	2 mov %rdi, (%rsi)
3 imul %rcx, %rax, %rdi	3 mov 8(%rsi), %rdi
4 mov %rsi, (%rcx)	4 add %rdi, %rcx
5 and %rcx, %rdi	5 mov %rdi, 16(%rsi)
6 mov %rcx, %rdi	6 mov %rcx, %rdi

Założmy, że procesor nie ma układów forwardowania danych ani wykrywania hazardów. Dla każdego z podanych fragmentów:

1. Dopisz dodatkowe instrukcje nop (nic nie rób, no operation), które zapewnią poprawne wykonanie.
2. Spróbuj usunąć hazardy za pomocą przestawiania/modyfikacji rozkazów, zmniejszając tym samym liczbę koniecznych rozkazów nop. Ewentualne modyfikacje rozkazów mogą polegać na zmianie rejestrów. Możesz założyć, że rejestry %r8 i %r9 nie są używane przez program.

Zadanie 3. Skompiluj poniższą funkcję i na podstawie wynikowego kodu maszynowego skonstruuj graf przepływu danych jak na rysunkach w [1, §5.7.3]. Wskaż w nim *ścieżkę krytyczną* (czyli taką, która zajmuje najwięcej czasu) na podstawie czasu opóźnienia przetwarzania instrukcji z tabeli [1, 5.12] (przyjmij, że mikro-operacje odczytu i zapisu do pamięci zajmują 5 cykli).

```
1 void nonsense(long a[], long k,  
2             long *dp, long *jp) {  
3     long e = a[2];  
4     long g = a[3];  
5     long m = a[4];  
6     long h = k * 11;  
7     long f = g * h;  
8     long b = a[f];  
9     long c = e * 8;  
10    *dp = m + c;  
11    *jp = b + 4;  
12 }
```

Uwaga! Zauważ, że w tym zadaniu ignorujemy ograniczenie na liczbę jednostek funkcyjnych wykonujących każdy typ operacji.

Zadanie 4. Załóżmy, że kod z poprzedniego zadania wykonuje się na procesorze out-of-order Core i7 z mikroarchitekturą Haswell, używaną na wykładzie i opisaną w [1, §5.7.1]. Procesor potrafi w jednym cyklu zegarowym zdekodować i zlecić do wykonania maksymalnie 4 instrukcje i ma osiem jednostek funkcyjnych (ang. *functional units*). Jak technika przezywania rejestrów (ang. *register renaming*) pomaga procesorowi w usuwaniu fałszywych zależności między instrukcjami? Ile cykli zegarowych zajmie mu wykonanie instrukcji z poprzedniego zadania?

Zadanie 5. Posługując się programem `llvm-mca` przedstaw symulację pojedynczego wywołania funkcji `nonsense`. Do parametrów programu dodaj: `-mcpu=haswell -timeline -iterations=1`. Na diagramie **Timeline view**² wskaż punkty wysłania (ang. *dispatch*), wykonania (ang. *execute*) i zatwierdzenia (ang. *retire*) instrukcji. Ile czasu zajmuje wykonanie funkcji według symulatora? Wyjaśnij z czego wynikają przestoje (ang. *stall*) w przetwarzaniu instrukcji?

Zadanie 6 (bonus). Posługując się programem `llvm-mca` powtórz pomiary widoczne na slajdzie 49 prezentacji z wykładu. Dokładniej rzecz biorąc:

- napisz fragment kodu zawierający L rozwinięć pętli i K akumulatorów,
- przetłumacz go do assemblera,
- wklej fragment odpowiadający za jedną iterację pętli do osobnego pliku (pozostałe instrukcje będą dla nas pomijalne),
- uruchom na nim program `llvm-mca`.

Doświadczenie wykonaj dla mnożenia liczb typu `double`, na procesorze z mikroarchitekturą Haswell (`-mcpu=haswell`). Przyjmij, że docelowa liczba elementów do wymnożenia wynosi 10 000. Możesz ograniczyć się do reprezentatywnego zbioru parametrów (K, L) , na przykład $(1, 1)$, $(1, 10)$, $(2, 10)$, $(4, 4)$, $(10, 10)$. Jeżeli Twoje wyniki nie są podobne do tych ze slajdu, zastanów się dlaczego. Czy jest różnica pomiędzy różnymi poziomami użytej optymalizacji (np. pomiędzy `-O0` a `-O2`)? Czy dobranie dużych wartości parametrów K i L mogłoby istotnie polepszyć/pogorszyć wydajność programu?

Literatura

- [1] „*Computer Systems: A Programmer's Perspective*”
Randal E. Bryant, David R. O'Hallaron; Pearson; 3rd edition, 2016
- [2] „*Modern Processor Design: Fundamentals of Superscalar Processors*”
John Paul Shen, Mikko H. Lipasti; McGraw-Hill; 1st edition, 2005

²<https://llvm.org/docs/CommandGuide/llvm-mca.html#timeline-view>